

Progressive Collaborative and Semantic Knowledge Fusion for Generative Recommendation

Longtao Xiao
Huazhong University of Science and
Technology
Wuhan, China
xiaolongtao@hust.edu.cn

Haozhao Wang
Huazhong University of Science and
Technology
Wuhan, China
hz_wang@hust.edu.cn

Cheng Wang
Huawei Technologies Ltd
Shenzhen, China
wangcheng250@huawei.com

Linfei Ji
Huazhong University of Science and
Technology
Wuhan, China
jilinfei@hust.edu.cn

Yifan Wang
Huazhong University of Science and
Technology
Wuhan, China
d202381481@hust.edu.cn

Jieming Zhu
Huawei Noah's Ark Lab
Shenzhen, China
jiemingzhu@ieee.org

Zhenhua Dong
Huawei Noah's Ark Lab
Shenzhen, China
dongzhenhua@huawei.com

Rui Zhang
Huazhong University of Science and
Technology
Wuhan, China
rayteam@yeah.net

Ruixuan Li
Huazhong University of Science and
Technology
Wuhan, China
rxli@hust.edu.cn

Abstract

With the recent surge in interest surrounding generative paradigms, generative recommendation has increasingly attracted the attention of researchers in the recommendation community. This paradigm generally consists of two stages. In the first stage, pretrained semantic embeddings or collaborative ID embeddings are quantized to create item codes, aiming to capture and preserve rich semantic or collaborative knowledge within these codes. The second stage involves utilizing these discrete codes to perform an autoregressive sequence generation task. **Existing generative recommendation methods often either overlook collaborative knowledge or semantic knowledge, or combine the two roughly.** In this paper, **we observe that naively concatenating representations from semantic modality and collaborative modality leads to a semantic domination issue, where the resulting representation is overly influenced by semantic information, effectively overshadowing the collaborative representation.** Consequently, downstream recommendation tasks fail to fully exploit the knowledge from both modalities, resulting in sub-optimal performance.

To address this, we propose a **progressive collaborative and semantic knowledge fusion model for generative recommendation**, named **PRORec**, which integrates semantic and collaborative knowledge **with a unified code** through a two-stage framework. Specifically, in the first stage, we propose a cross-modality knowledge

alignment task during the pre-training of the non-generative recommendation model, which integrates semantic knowledge into collaborative embeddings, enhancing their representational capability. In the second stage, we propose an in-modality knowledge distillation task, designed to encourage the generative recommendation model to effectively capture and integrate knowledge from both semantic and collaborative modalities. Extensive experiments on three widely used benchmarks validate the effectiveness of our approach, demonstrating its superiority compared to existing methods.

CCS Concepts

• Information systems → Recommender systems.

Keywords

Generative Recommendation, Autoregressive Generation, Semantic Tokenization, Collaborative-Semantic Fusion

ACM Reference Format:

Longtao Xiao, Haozhao Wang, Cheng Wang, Linfei Ji, Yifan Wang, Jieming Zhu, Zhenhua Dong, Rui Zhang, and Ruixuan Li. 2025. Progressive Collaborative and Semantic Knowledge Fusion for Generative Recommendation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 11 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Recommendation systems have become widely adopted in modern society, such as in video [44], movie [6], and music [16] scenarios. Modern recommendation systems typically adopt a retrieve-and-rank strategy. During the retrieval process, the model needs to retrieve the most relevant items from a large candidate pool based on the item embeddings learned through representation learning. However, existing approaches, whether based on dual-tower models [4, 35] or graph models [34, 40], generally rely on dot-product

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
<https://doi.org/XXXXXXX.XXXXXXX>

(cosine) similarity with approximate nearest neighbor (ANN) search [1, 2]. Although ANN accelerates top-k recommendation retrieval, its search indices are constructed using tools such as Faiss [14] and SCANN [8], which are independent of the optimization process of the recommendation model, thereby limiting the overall effectiveness of recommendation systems [50].

To address this limitation, generative recommendation [22, 29, 36, 37], which adopts code as the core component and frames the recommendation task as autoregressive code sequence generation, has emerged as a promising research direction owing to its potential to enable more efficient decoding without the need for ANN indexes. Unlike traditional methods that match users and items based on embeddings, generative recommendation predicts candidate item code directly, as depicted in Figure 1. Specifically, pre-trained semantic encoders [19, 26] or collaborative encoders [15, 46] are first utilized to encode item semantic or ID information into embeddings. Then, embedding quantization [18, 28] is applied to map these embeddings into discrete codes forming a lookup table of item codes. The input of transformer will be converted into a sequence of codes, enabling the model to autoregressively predict the next item's code step by step.

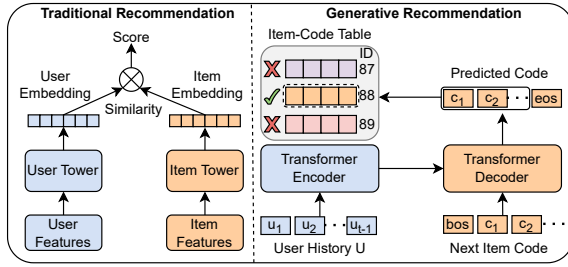


Figure 1: Traditional vs. Generative Recommendation.

Existing methods typically construct independent codes for different modalities knowledge, either collaborative or semantic modality. For instance, Recforest [7] encodes item collaborative knowledge into a single set of codes, whereas TIGER [29] focuses on encoding item semantic knowledge into a separate code set. Furthermore, **EAGER [39] encodes both collaborative and semantic knowledge into two independent sets of codes.** These approaches have demonstrated that leveraging both collaborative and semantic information is necessary to enhance the effectiveness and robustness of recommendation models. **However, for generative recommenders aiming to integrate both semantic and collaborative knowledge, two separate codes are unacceptable, as the dual-code framework significantly increases storage and inference costs, rendering it impractical for large-scale deployments. Besides, the intrinsic misalignment between semantic and collaborative knowledge limits the potential to fully exploit their complementary strengths with two separate codes [17].**

To overcome the aforementioned challenges, we propose a novel method, PRORec, which integrates collaborative and semantic knowledge into a unified code for generative recommendation. Unified codes not only reduce storage and computation demands but also

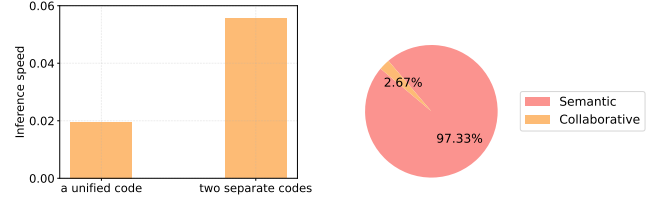


Figure 2: Comparison of inference speed (second per sample, topk=5, beam size=100) between a unified code and two separate codes setups on the Beauty dataset.

Figure 3: Proportional similarity of semantic modality and collaborative modality to the final representation with concatenation method on the Beauty dataset.

accelerate inference and lower latency, which are critical for real-time responses. **As illustrated in Figure 2, a unified code demonstrates significantly 2.8x faster inference compared to two separate codes. Moreover, dual-code methods inherently treat semantic and collaborative knowledge as independent entities, thereby missing opportunities to harness their synergistic potential. In contrast, a unified code, by integrating both knowledge at an early stage, can more effectively exploit their complementary strengths.**

However, it is non-trivial to adopt a unified code. The core challenge in adopting a unified code lies in how to get a unified representation which integrates knowledge from both collaborative and semantic modalities. A common practice is to concatenate semantic and collaborative features to create a unified representation. However, this approach suffers from a phenomenon termed *the semantic dominance issue*. As depicted in Figure 3, the final representation aligns more closely with the semantic side, leading to underutilization of the collaborative side. This imbalance not only limits the potential of integrating collaborative knowledge but, in extreme cases, can even degrade performance to levels below those achieved with semantic knowledge alone. The root cause of this issue lies in the inability of concatenation to account for the inherent differences between the two different modalities, such as variations in signal strength and representational misalignment [38]. Consequently, semantic representations dominate the final output, undermining the expected advantages of unified representations.

We analyze the challenges of integrating both collaborative and semantic knowledge into a unified code for generative recommendation and address them from the following three aspects:

Firstly, **a unified code** that encodes both collaborative and semantic knowledge is crucial for practical deployment. Due to the inherent modality differences [17, 38], adopting a unified code poses challenges. Therefore, we propose a two-stage knowledge integration framework. Specifically, in the first stage, it uses Llama2-7b [33] to extract semantic knowledge and DIN [46] to extract collaborative knowledge, then adaptively learn a unified representation which integrates both knowledge to construct discrete item codes, referred to as Unicodes, which serve as item identifiers for the second stage. In the second stage, our architecture consists of an encoder that models the user's interaction history and a decoder that predicts the unicode of the target item.

Secondly, to **resolve the semantic domination issue**, we propose a learnable modality adaption layer and a cross-modality knowledge alignment task, which help to adaptively learn an integrated embedding through the joint optimization with next item prediction task. By providing fine-grained contrastive supervision signals, it encourages the representation to leverage both collaborative and semantic knowledge more balanced, avoiding the overemphasis on semantic modality.

Thirdly, ensuring **comprehensive and sufficient learning** is essential for fully exploiting both knowledge. To compensate for the information loss caused by the quantization process, we introduce an intra-modality knowledge distillation task with a special token in the second stage. Inspired by contrastive learning paradigms and the success of the [CLS] token in BERT [5] for classification tasks, we design a special token to summarize the sequence’s knowledge. The embedding of this token is then used in a global contrastive task against the target item integrated embedding, enabling the model to distill the integrated knowledge.

In summary, our contributions can be concluded as follows:

- We introduce PRORec, with a unified code, Unicodes, which integrates both collaborative and semantic knowledge for generative recommendation. By utilizing the unified code, we achieve faster inference while requiring only half the storage space compared to two separate codes setup.
- We propose a learnable adaption layer and a joint optimization of cross-modality knowledge alignment and next item prediction tasks to adaptively learn an integrated embedding for resolving the semantic domination issue, and introduce an intra-modality knowledge distillation task to ensure comprehensive and sufficient learning, compensating for the information loss.
- Extensive experiments on three public recommendation benchmarks demonstrate PRORec’s superiority over existing methods, encompassing both generative and traditional paradigms. Additionally, we observe that PRORec demonstrates the scaling law characteristic.

2 RELATED WORK

Sequential recommendation models focus on modeling user behavior as a chronologically ordered sequence of interactions, aiming to predict the next item a user will engage with.

Traditional Approaches. Early approaches primarily utilized Markov Chains (MCs) [10, 30] to capture the transition probabilities between items. With the rise of deep learning, an ID-based recommendation paradigm emerged. Various deep neural network architectures have been developed under this paradigm. For instance, GRU4Rec [12] was the first to employ GRU-based RNNs for sequential recommendations, while SASRec [15] introduced a self-attention mechanism similar to decoder-only transformer models, to capture long-range dependencies. Inspired by the success of masked language modeling, BERT4Rec [31] utilized transformers with masking strategies to enhance sequential recommendation tasks. Building on the masking technique, S³-Rec [47] learns the correlations among attributes, items, subsequences, and sequences through the principle of mutual information maximization (MIM), thereby enhancing data representation.

Generative Approaches. Unlike traditional embedding-based methods, which rely on dot-product (cosine) similarity and external ANN search systems for top-k retrieval, generative approaches predict item identifiers directly. Generative methods can be broadly categorized into two types: prompt fine-tuning strategies based on *off-the-shelf* large language models (LLMs) and training from scratch for custom-designed models.

For LLMs based methods [13, 17, 20, 23, 45], the focus is on designing refined prompts and fine-tuning tasks that help language models better understand recommendation tasks. LC-Rec [45] introduces a learning-based vector quantization approach for semantically meaningful item indexing and fine-tuning tasks that align collaborative signals with LLM representations, achieving superior performance in diverse scenarios. CCF-LLM [23] transforms user-item interactions into hybrid prompts encoding both semantic knowledge and collaborative signals, utilizing an attentive cross-modal fusion strategy to integrate embeddings from different modalities. SC-Rec [17] utilizes multiple item indices and prompt templates, alongside a self-consistent re-ranking mechanism, to more effectively merge collaborative and semantic knowledge.

For methods that train from scratch [7, 21, 29, 38, 39, 41, 48, 49], the primary focus is on converting the raw sequence recommendation task into an autoregressive generation task. Tree-based methods [7, 48, 49], such as RecForest [7], have shown promising performance by constructing multiple trees and integrating transformer-based structures for routing. Additionally, TIGER [29] introduced the concept of semantic IDs, representing each item as a set of tokens derived from its side information, and predicting next item tokens in a sequence-to-sequence manner. EAGER [39] employs a dual-stream generative framework to parallelly utilize semantic and behavioral information with two separate codes, generating recommended items from each respective pipeline, and ultimately selecting the top-k items based on confidence scores.

In this paper, we aim to seamlessly integrate collaborative and semantic modality knowledge into a unified code for generative recommendation, while ensuring no additional computational and storage overhead compared with solely utilizing one modality.

3 METHOD

3.1 Problem Formulation

Given an item corpus I and a user’s historical interaction sequence $U = [u_1, u_2, \dots, u_{t-1}]$ where $u \in I$, the target of a sequential recommendation system is to predict the next most likely item $u_t \in I$ that the user may interact with.

In the generative framework, each item u is represented by a sequence of codes $C = [c_1, c_2, \dots, c_L]$, where L denotes the length of the code sequence. Here, the sequential recommendation task shifts to predicting the codes C_t of the next item u_t based on the user’s historical interaction sequence U . During training, the model first encodes the user’s historical interaction sequence U and then generates the codes C_t of the target item u_t step by step at the decoder. The decoding process is defined by the following formula:

$$p(C_t|U) = \prod_{i=1}^L p(c_i|U, c_1, c_2, \dots, c_{i-1}) \quad (1)$$

During the inference phase, the decoder performs beam search to autoregressively generate the codes of the top-k items.

3.2 Overall Pipeline

We present PRORec overall framework in Figure 4, which consists of two stages. In the first stage, we first adaptively learn integrated embeddings through the joint optimization of proposed cross-modality knowledge alignment task and next item prediction task, which fuse knowledge from both collaborative and semantic modalities. Subsequently, the integrated embeddings are quantized into discrete unicodes that act as item identifiers in the second stage. The second stage is the generative recommendation phase, comprising an encoder, a decoder and an intra-modality knowledge distillation task to compensate for the information loss caused by the quantization process. Here, the user's historical interaction sequence is transformed into unicodes sequence using the item-unicode mapping table obtained in the first stage. The encoder models the user's historical interactions to capture their interests, which are then passed into the decoder to guide the generation of unicodes for candidate items. Once training is complete, we select the top-k recommendations based on confidence scores.

3.3 Stage I: Item Unicodes Generation

In this stage, our objective is to construct item unicodes which are capable of encoding both collaborative and semantic knowledge while utilizing a single unified codebook. Let X and S denote the ID sequence and corresponding item semantic side information (e.g. such as titles) of the user's historical interaction sequence U , respectively. Then, we first use a randomly initialized sequential recommendation model (e.g. DIN [46]) to encode the user's historical sequence X into collaborative embeddings E_C , and employ a pretrained semantic encoder (e.g. Llama2-7b [33]) to encode items' semantic knowledge S into embeddings E_S .

Modality adaption layer. To bridge the modality gap between two modalities, we propose using a learnable modality adaptation layer with AdaLN that maps the semantic embeddings E_S into the same embedding space as collaborative embeddings. AdaLN [27] introduces learnable parameters to adaptively adjust the normalization process according to the input, effectively capturing the distinctive characteristics of the two different modalities. The mapping process is defined as follows:

$$E_T = \text{MAL}(E_S) = \text{AdaLN}(WE_S + b) \quad (2)$$

where MAL refers to the modality adaptation layer, with W and b as learnable parameters, and AdaLN indicating the adaptive normalization layer.

Cross-modality knowledge alignment task. To effectively integrate semantic and collaborative knowledge, we introduce a cross-modality knowledge alignment task designed to align embeddings from the collaborative modality E_C and semantic modality E_T . For a given item $i \in I$, our objective is to bring its collaborative embedding E_{C_i} closer to its semantic embedding E_{T_i} for item i while distancing E_{C_i} from the semantic embedding E_{T_j} of a different item $j \in I$, where j is an in-batch negative sample selected from the same batch as item i . By structuring the alignment in this way, the learned embedding for item i is able to encapsulate knowledge from

both modalities. To achieve this goal, we adopt the Info-NCE loss [3]. The loss function for our cross-modality knowledge alignment task is defined as follows:

$$\mathcal{L}_{\text{align}} = -\log \sum_{i \in I} \frac{\exp(\text{sim}(E_{C_i}, E_{T_i})/\tau)}{\sum_{\substack{j \in \text{in-batch neg-samples} \\ j \neq i}} \exp(\text{sim}(E_{C_i}, E_{T_j})/\tau)} \quad (3)$$

where $\text{sim}(\cdot, \cdot)$ represents a similarity function between embeddings (e.g. dot product or cosine similarity), and τ is a temperature parameter used to control the smoothness of the similarity.

Next item prediction task. In addition to the alignment loss described above, we adopt next item prediction task to enhance the model's ability to capture the sequence dependency, thereby supporting the learning of robust collaborative representations. This task takes the user's historical interaction sequence $[x_1, x_2, \dots, x_{t-1}]$ as input, learns a representation of the user's preferences, and computes matching scores by measuring the similarity between the learned preferences and candidate items. These scores are subsequently normalized to derive the probability distribution over the target item. The goal is to maximize the probability assigned to the actual target item x_t , which can be formally represented as follows:

$$\mathcal{L}_{\text{seq}} = -\sum_{t=2}^L \log p(x_t | x_1, x_2, \dots, x_{t-1}) \quad (4)$$

Joint optimization. In the first stage, we jointly optimize the two losses to train the modality adaption layer and sequential recommendation model. The total loss function for the first stage is defined as follows:

$$\mathcal{L}_{\text{Stage I}} = \mathcal{L}_{\text{seq}} + \alpha \mathcal{L}_{\text{align}} \quad (5)$$

where $\mathcal{L}_{\text{seq}}$ is the next item prediction loss defined in equation (3), $\mathcal{L}_{\text{align}}$ is the cross-modality knowledge alignment loss defined in equation (4), and α is a tunable hyperparameter used to adjust the relative importance of the alignment loss.

Unified Codes. After the training is finished, following previous work [39], we apply the widely-used hierarchical k-means algorithm to the integrated embeddings for quantization. In this process, the embeddings are initially partitioned into K clusters. Each cluster is then recursively subdivided into K child clusters until each child cluster contains a single item. Finally, for each item, the cluster indices at each hierarchical level are recorded sequentially, forming a list that serves as the item's unicode.

3.4 Stage II: Generative Recommendation

Encoding process. On the encoder side, given a user's interaction history $X = [x_1, x_2, \dots, x_{t-1}]$, we input it into an encoder consisting of stacked multi-head self-attention layers and feed-forward layers, following the Transformer architecture. The encoder processes X to produce a feature representation H , which captures the user's interests and will be passed to the decoder.

Decoding process. On the decoder side, items are mapped to their unicodes derived in Stage I. Our objective changes from predicting the next item x_t to predicting its item unicode $[c_1, c_2, \dots, c_L]$. For training, a special token $\langle \text{BOS} \rangle$ is prepended to the item unicode to construct the decoder input. The generative recommendation loss

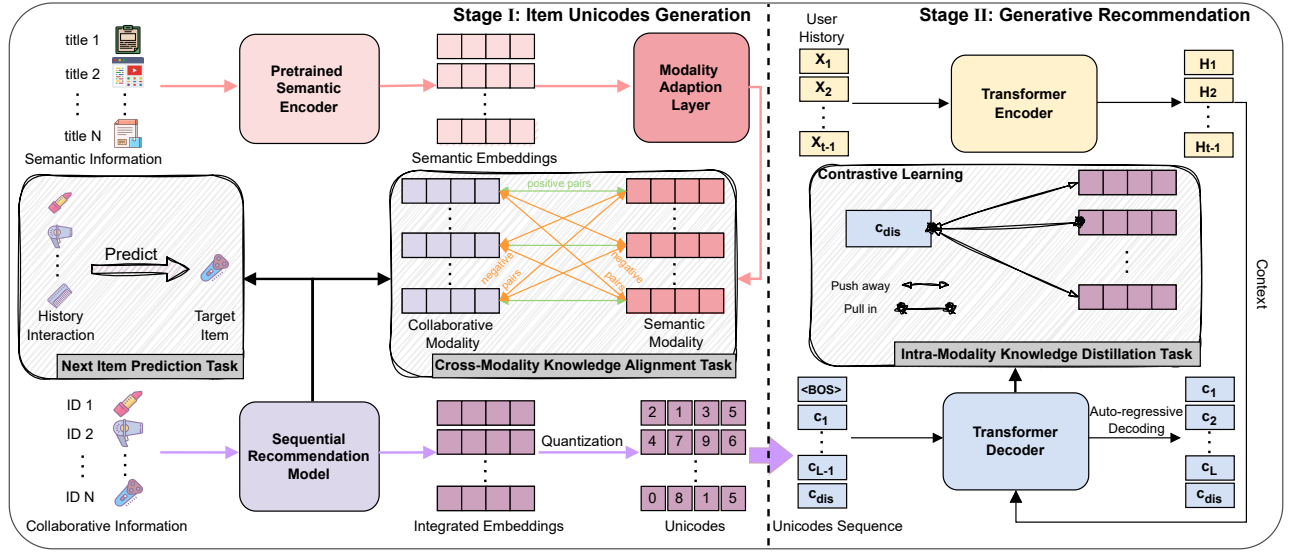


Figure 4: An overview of PRORec. PRORec consists of two stages. The first stage integrates semantic and collaborative knowledge to construct the unified code, Unicode for each item. The second stage utilizes the obtained unicodes to perform generative recommendation. To achieve the goal of utilizing a unified code to encode the two different knowledge for generative recommendation, we introduce two auxiliary tasks at each stage: a cross-modality knowledge alignment task (CKA) in the first stage and an intra-modality knowledge distillation task (IKD) in the second stage. Besides, in the first stage, a modality adaption layer with AdaLN is also introduced to bridge the modality gap between semantic and collaborative space.

is computed using a cross-entropy loss function, defined as follows:

$$\mathcal{L}_{\text{gen}} = \sum_{i=1}^L \log p(c_i | x, \langle \text{BOS} \rangle, c_1, \dots, c_{i-1}), \quad (6)$$

Intra-modality knowledge distillation task. To compensate for the information loss caused by the quantization process in the first stage, we introduce an intra-modality knowledge distillation task to distill the global knowledge. Inspired by the use of the [CLS] token in BERT for capturing global context in classification tasks, we append a learnable token $[c_{\text{dis}}]$ to the end of the decoder input, allowing gradients to propagate through each preceding unicode. This design encourages $[c_{\text{dis}}]$ to capture global information.

Specifically, the final layer output corresponding to $[c_{\text{dis}}]$ is used in a global contrastive learning objective over the item corpus: the positive sample is the integrated embedding E_t of item x_t learned from the first stage, and the negative sample is a randomly selected integrated embedding E_{neg} from other items in the corpus, excluding x_t . This objective pulls the $[c_{\text{dis}}]$ final layer output closer to the positive sample E_t and pushes it away from the negative sample E_{neg} . The loss function for the distillation task is defined as follows:

$$\mathcal{L}_{\text{distillation}} = -\log \frac{\exp(c_{\text{dis}} \cdot E_t)}{\exp(c_{\text{dis}} \cdot E_t) + \sum \exp(c_{\text{dis}} \cdot E_{\text{neg}})} \quad (7)$$

where c_{dis} denotes the final layer output of the decoder for the special token $[c_{\text{dis}}]$; E_{neg} is the integrated embedding for a negative sample as learned in Stage I, while E_t represents the integrated embedding of the target item, also learned in Stage I.

Joint optimization. Finally, in Stage II, we jointly optimize these two loss functions. The overall loss function for Stage II is defined as follows:

$$\mathcal{L}_{\text{Stage II}} = \mathcal{L}_{\text{gen}} + \beta \mathcal{L}_{\text{distillation}} \quad (8)$$

where $\mathcal{L}_{\text{gen}}$ is the generative recommendation loss, $\mathcal{L}_{\text{distillation}}$ is the intra-modality knowledge distillation loss, and β is a hyperparameter that balances the two objectives.

3.5 Training and Inference

Training. we employ a two-stage training process. In the first stage, we use DIN as the backbone and optimize the model with loss $\mathcal{L}_{\text{stage I}}$. After the first stage training is complete, we extract the item integrated embeddings and derive item unicodes. In the second stage, we use a Transformer for generative recommendation, optimizing it with loss $\mathcal{L}_{\text{stage II}}$.

Inference. During inference phase, we rely exclusively on the Transformer model. In the decoding process, beam search is applied iteratively to generate each token within the item unicode sequence. Once finished, the item unicodes are mapped to their corresponding items through the item-unicode lookup table, creating a ranked recommendation list based on confidence scores to produce the final top- k results.

3.6 Computational and Storage Costs Analysis

In our approach, we employ a unified code to encode both semantic and collaborative information. Regarding computational costs, the encoder processes the user interaction sequence only once, leaving

the main computational burden to the decoder, which iteratively generates each token of the item code. Assuming the code length is fixed and decoding an item code requires $O(n)$ time, existing methods [17, 39] that use separate codes for each modality require K parallel decoders to predict codes for all modalities. Furthermore, these methods involve a confidence-based ranking step to derive the final recommendation. Excluding the computational cost of this ranking, their decoding cost is $O(Kn)$, where K is the number of modalities. In contrast, our method, with its single unified code, achieves a decoding cost of only $O(n)$.

For storage costs, assuming the storage requirement for one code is $O(m)$, existing methods require $O(Km)$ space to store codes for all modalities. By contrast, our method requires only $O(m)$ space to store multi-modal information within a single unified code.

Table 1: Comparison of our method with typical generative recommendation methods TIGER and EAGER on computational and storage costs.

	TIGER	EAGER	PRORec (Ours)
Computation Cost	$O(n)$	$O(2n)$	$O(n)$
Storage Cost	$O(m)$	$O(2m)$	$O(m)$
Used Modality	Semantic Only	Semantic + Collaborative	Semantic + Collaborative

Table 2: Statistics of the Datasets.

Dataset	#Users	#Items	#Interactions	#Density
Beauty	22,363	12,101	198,360	0.00073
Sports and Outdoors	35,598	18,357	296,175	0.00045
Toys and Games	19,412	11,924	167,526	0.00073

4 EXPERIMENTS

We analyze the proposed PRORec method and demonstrate its effectiveness by answering the following research questions:

- **RQ1:** How does PRORec perform compared with existing best-performing sequential recommendation methods among different datasets?
- **RQ2:** How effective is PRORec in mitigating the semantic domination issue?
- **RQ3:** Do cross-modality knowledge alignment and intra-modality knowledge distillation tasks each contribute positively to PRORec's performance?
- **RQ4:** How do different ablation variants and hyper-parameter settings affect the performance of PRORec?

4.1 Experimental Setting

Dataset. We conduct experiments on three public benchmarks commonly used in the sequential recommendation task. For all datasets, we group the interaction records by users and sort them by the interaction timestamps ascendingly. Following [30, 43], we only keep the 5-core dataset, which filters unpopular items and inactive users with fewer than five interaction records. Statistics of these datasets are shown in Table 2.

- **Amazon:** Amazon Product Reviews dataset [25], containing user reviews and item metadata from May 1996 to July 2014. In particular, we use three categories of the Amazon Product Reviews dataset for the sequential recommendation task: "Beauty", "Sports and Outdoors", and "Toys and Games".

Evaluation Metrics. We employ two broadly used criteria for the matching phase, *i.e.*, Recall and Normalized Discounted Cumulative Gain (NDCG). We report metrics computed on the top 10/20 recommended candidates. Following the standard evaluation protocol [15], we use the leave-one-out strategy for evaluation. For each item sequence, the last item is used for testing, the item before the last is used for validation, and the rest is used for training. During training, we limit the number of items in a user's history to 20.

Implementation Details. In our experiments, we set the number of encoder layers to 1 and the number of decoder layers to 4. Following previous work [7, 39], we adopt DIN as our sequential recommendation model and use pretrained Llama2-7b as our semantic encoder, with the hidden size set to 128 as reported in [29, 39]. The number of clusters in the hierarchical k-means is set to 256. To train our model, we use the Adam optimizer with a learning rate of 0.001 and apply a warmup strategy for stable training. The batch size is set to 256. Notably, PRORec demonstrates robustness to the hyperparameters of the CKA and IKD tasks, attributed to their fast convergence. Consequently, the loss coefficients α and β are both set to 1.

4.2 Performance Comparison (RQ1)

Baselines. We compare PRORec with several representative related baselines, encompassing classical sequential modeling approaches as well as the latest emergent generative techniques.

(1) For *classical sequential* methods, we have:

- **GRU4REC** [11] is an RNN-based model that utilizes Gated Recurrent Units (GRUs) to model user click sequences.
- **Caser** [32] is a CNN-based method capturing high-order Markov Chains by modeling user behaviors through both horizontal and vertical convolutional operations.
- **SASRec** [15] is a self-attention-based sequential recommendation model that utilizes a unidirectional Transformer encoder with a multi-head attention mechanism to effectively model user behavior and predict the next item in a sequence.
- **BERT4Rec** [31] adopts a Transformer model with the bidirectional self-attention mechanism and uses a Cloze objective loss for the modeling of item sequences.
- **HGN** [24] adopts hierarchical gating networks to capture long-term and short-term user interests.
- **FDSA** [43] leverages self-attention networks to separately model item-level and feature-level sequences, emphasizing the transformation patterns between item features and utilizing a feature-level self-attention block to capture feature transition dynamics.
- **S³-Rec** [47] employs mutual information maximization to pre-train a bi-directional Transformer for sequential recommendation, enhancing the model's ability to learn correlations between items and their attributes through self-supervised tasks.

(2) For *generative* methods, we have:

- **RecForest** [7] jointly learns latent embeddings and indices through multiple K -ary trees, utilizing hierarchical balanced clustering

Table 3: Performance comparison of different methods. The best performance is highlighted in bold while the second best performance is underlined. All the results of PRORec are statistically significant with $p < 0.05$ compared to the best baseline models.

Dataset	Metric	Classical							Generative			PRORec (Ours)
		GRU4REC	Caser	SASRec	BERT4Rec	HGN	FDSA	S ³ -Rec	Recoreast	TIGER	EAGER	
Beauty	Recall@10	0.0283	0.0347	0.0605	0.0347	0.0512	0.0407	0.0647	0.0664	0.0617*	<u>0.0836</u>	0.0939
	Recall@20	0.0479	0.0556	0.0902	0.0599	0.0773	0.0656	0.0994	0.0915	0.0924*	<u>0.1124</u>	0.1289
	NDCG@10	0.0137	0.0176	0.0318	0.0170	0.0266	0.0208	0.0327	0.0400	0.0339*	<u>0.0525</u>	0.0559
	NDCG@20	0.0187	0.0229	0.0394	0.0233	0.0332	0.0270	0.0414	0.0464	0.0417*	<u>0.0599</u>	0.0646
Sports	Recall@10	0.0204	0.0194	0.0350	0.0191	0.0313	0.0288	0.0385	0.0247	0.0376*	<u>0.0441</u>	0.0471
	Recall@20	0.0333	0.0314	0.0507	0.0315	0.0477	0.0463	0.0607	0.0375	0.0577*	<u>0.0659</u>	0.0710
	NDCG@10	0.0110	0.0097	0.0192	0.0099	0.0159	0.0156	0.0204	0.0133	0.0196*	<u>0.0236</u>	0.0259
	NDCG@20	0.0142	0.0126	0.0231	0.0130	0.0201	0.0200	0.0260	0.0164	0.0246*	<u>0.0291</u>	0.0319
Toys	Recall@10	0.0176	0.0270	0.0675	0.0203	0.0497	0.0381	0.0700	0.0383	0.0578*	<u>0.0714</u>	0.0822
	Recall@20	0.0301	0.0420	0.0941	0.0358	0.0716	0.0632	0.1065	0.0483	0.0838*	<u>0.1024</u>	0.1154
	NDCG@10	0.0084	0.0141	0.0374	0.0099	0.0277	0.0189	0.0376	0.0285	0.0321*	0.0505	<u>0.0489</u>
	NDCG@20	0.0116	0.0179	0.0441	0.0138	0.0332	0.0252	0.0468	0.0310	0.0386*	<u>0.0538</u>	0.0573

and a transformer-based encoder-decoder routing network to enhance accuracy and memory efficiency in identifying top-n items.

- **TIGER** [29] utilizes a pre-trained T5 encoder to learn semantic identifiers for items, autoregressively decodes target candidates using these identifiers, and incorporates RQ-VAE quantization to build generalized item identifiers without relying on sequential order.
- **EAGER** [39] integrates behavioral and semantic information through a two-stream architecture with shared encoding, separate decoding, and strategies for contrastive and semantic-guided learning to enhance collaborative information utilization.

Results. Table 3 reports the overall performance of three datasets. The results for all baselines without the superscript * are taken from the publicly accessible results [39, 47]. Due to missing values for TIGER, we reimplement it and report our experimental results. From the results, we have the following observations:

- **PRORec almost achieves better results than base models among different datasets.** Specifically, on the Beauty dataset, compared to the second best baseline EAGER, we achieve improvements of 14.68% and 7.85% in *Recall@20* and *NDCG@20*, respectively, which represents a significant improvement in the recommendation field. We attribute the improvements to PRORec’s effective exploitation of the complementary strengths of collaborative and semantic knowledge with a unified code, which allows it to achieve superior performance relative to EAGER, despite utilizing approximately half the number of total parameters.
- **PRORec beats the previous generative models on most datasets.** Compared to previous generative models, PRORec employs a two-stage multi-task joint optimization framework to

integrate collaborative and semantic knowledge with a unified code, which successfully captures the interplay between these two types of knowledge, allowing for a more thorough and efficient exploitation of their complementary strengths. The improvements validate the effectiveness of our method and demonstrate the necessity of a unified code.

- **In most cases, generative models outperform classical models.** Unlike traditional sequence modeling approaches, which use non-informative item IDs as identifiers, generative models assign hierarchical codes to items as identifiers. This hierarchical structure inherently encodes prior knowledge, facilitating a more effective alignment with the token-by-token generation framework of the Transformer decoder.

Table 4: Proportional similarity of the semantic modality and collaborative modality to the final representation across different approaches.

Approach	Semantic modality	Collaborative modality
Concat	97.33%	2.67%
Ours	59.89%	40.11%

4.3 Semantic Domination Issue (RQ2)

To investigate whether our method addresses the semantic dominance issue, we conduct experiments on the Amazon Beauty dataset. We first extract the semantic embeddings of the items, then use only the item IDs from historical interaction data and apply the classical DIN model to obtain the collaborative embeddings. Following the

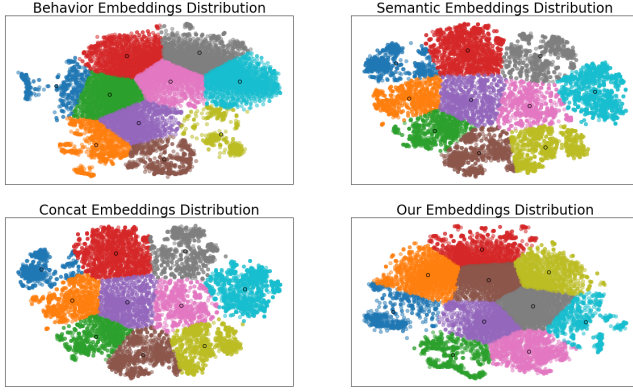


Figure 5: T-SNE visualization of different embeddings distribution after applying K-means clustering (clusters=10).

Table 5: Performance of different methods.

Dataset	Beauty			
Metric	Recall@10	Recall@20	NDCG@10	NDCG@20
Semantic Only	0.0810	0.1102	0.0498	0.0571
Collaborative Only	0.0803	0.1095	0.0495	0.0569
Concat	0.0807	0.1085	0.0504	0.0574
Ours	0.0939	0.1289	0.0559	0.0646

common approach of fusing knowledge from different modalities, we concatenate the semantic embeddings with the collaborative embeddings. Notably, to mitigate the issue caused by the large dimensional differences between embeddings from the two modalities, we first apply PCA to reduce the dimension of the semantic embeddings to match that of the collaborative embeddings, and then concatenate them to obtain the concatenation embeddings. In addition, we obtain the integrated embeddings of items using our proposed method. We use T-SNE to visualize the distribution of these different embeddings. To more clearly illustrate the differences in their distributions, we apply K-means clustering algorithm to these embeddings, using different colors to represent data points from each cluster, as shown in Figure 5.

Furthermore, to quantitatively measure the relative similarity of semantic embeddings and collaborative embeddings on the final embeddings using different methods, we use KL divergence to assess the similarity of their distribution. The relative similarity of each modality is then calculated based on the KL divergence values. The formula for above process is as follows:

$$\text{Similarity}_{\text{semantic}} = 1 - \frac{\text{KL}(S \parallel E)}{\text{KL}(S \parallel E) + \text{KL}(C \parallel E)} \quad (9)$$

$$\text{Similarity}_{\text{collaborative}} = 1 - \frac{\text{KL}(C \parallel E)}{\text{KL}(S \parallel E) + \text{KL}(C \parallel E)} \quad (10)$$

where E represents the final embeddings using different methods, S denotes the semantic embeddings, and C refers to the collaborative embeddings.

As shown in Table 4, the concatenation method exhibits a significant disparity in similarity proportions between semantic and

Table 6: Performance comparison of different variants.

Dataset	Metric	\	CKA	CKA + IKD (Ours)
Beauty	Recall@10	0.0807	0.0827	0.0939
	Recall@20	0.1085	0.1122	0.1289
	NDCG@10	0.0504	0.0509	0.0559
	NDCG@20	0.0574	0.0583	0.0646

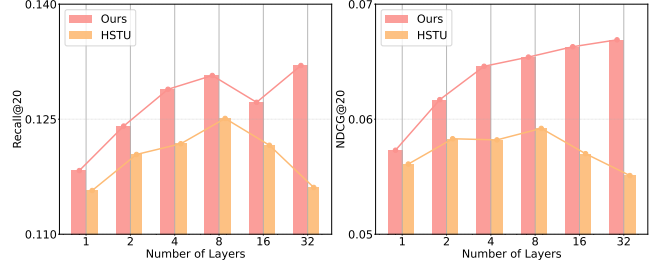


Figure 6: Impact of the number of model layers on the Beauty dataset. The number of layers varies from 1 to 32.

collaborative modalities, and Figure 5 further demonstrates that the embeddings obtained by the concatenation method are highly similar to the distribution of semantic embeddings. These observations confirm the presence of the semantic dominance problem. The traditional concatenation method tends to overly emphasize semantic knowledge while greatly ignoring collaborative knowledge, which poses a substantial obstacle to downstream recommendation effectiveness. In contrast, our proposed method effectively addresses this issue. It not only learns embeddings from both collaborative and semantic modalities in a more balanced manner at the distributional level but also achieves a more proportional similarity from each modality in quantitative evaluations. This prevents the semantic embeddings from dominating the final representation. As illustrated in Table 5, our method significantly outperforms both single-modality approaches and the concatenation-based fusion method, achieving the best results across all evaluation metrics. Interestingly, the concatenation method performs worse than the semantic-only approach in Recall@10 and Recall@20, indicating a negative impact on performance. This further highlights the severity of the semantic dominance problem and underscores the advantages of our proposed approach. We attribute these improvements to the successful integration of collaborative and semantic knowledge, which, unlike the concatenation method, better captures the complementary nature of the two different knowledge, enabling a more balanced and effective fusion of information from both modalities.

4.4 Ablation Study (RQ3)

We conducted a thorough evaluation of the contribution of each component in PRORec. Specifically, we incrementally removed the intra-modality knowledge distillation (IKD) task and the cross-modality knowledge alignment (CKA) task to create the ablation architectures. The results in Table 6 reveal the following insights:

- Removing either the CKA or IKD task degrades the model’s performance, while omitting both results in the worst outcome (i.e., the baseline model obtained by simply concatenating semantic and collaborative embeddings). These findings confirm the effectiveness of the two proposed tasks.
- The introduction of the CKA task enhances recommendation performance, as it effectively integrates knowledge from both collaborative and semantic modalities. By leveraging their complementary characteristics, the CKA task enables the model to learn more expressive and informative item representations.
- Incorporating both the CKA and IKD tasks allows the model to achieve state-of-the-art performance. Building on the foundation established by the CKA task, the IKD task further promotes the learning of integrated knowledge by encouraging the model to better exploit the complementary strengths of collaborative and semantic knowledge. The observation also highlights the crucial role of both tasks in enabling the model to acquire more powerful unified codes of items.

In conclusion, each component of PRORec is essential to enhancing the effectiveness of generative recommendation.

4.5 Scaling Law Study (RQ4)

Generative models have gained considerable attention due to their potential to exhibit scaling law properties. However, research exploring this phenomenon in the context of generative recommendation remains limited. Although recent study like HSTU [42] has examined its scaling law characteristics, they are based solely on collaborative information derived from pure item IDs. To the best of our knowledge, no studies have investigated the scaling law of models that integrate both semantic and collaborative information. In this work, we are the first to explore this and compare our method with HSTU to analyze the scaling law differences between the two approaches.

As shown in Figure 6, while HSTU demonstrates scaling law on the MovieLens-20M dataset in [42], it exhibits performance stagnation on the Amazon Beauty dataset once the model reaches eight layers. Beyond this point, the model’s performance begins to decline as the number of layers increases, a trend consistent with the observations in Table 1 of [9] on the Amazon Book dataset. In contrast, our method continues to show performance improvements on the Beauty dataset as the number of layers increases. Furthermore, our method consistently outperforms HSTU across all configurations, achieving a significant improvement of up to 21.42% in NDCG@20, demonstrating its superiority and effectiveness.

We attribute the superior performance of our method to its ability to simultaneously incorporate both collaborative and semantic knowledge with a unified code, effectively capturing their complementary nature. As a result, deeper models demonstrate enhanced capability in integrating these two types of knowledge, thus leading to consistent performance improvements.

4.6 Hyper-Parameter Analysis (RQ4)

Choice of different semantic encoders. To further investigate the influence of different pre-trained semantic encoders on recommendation performance, we conducted additional experiments on the Beauty dataset. Specifically, we employed various pre-trained

semantic encoders to extract item semantic knowledge and evaluated their effectiveness using our proposed method. As presented in Figure 7, the results reveal the following insights: (1) Llama2-7b achieved the best performance, likely due to its substantially larger training corpus compared to Sentence-T5 and BERT. This extensive corpus enables Llama2-7b to produce richer semantic embeddings, which allow the unicones to capture more comprehensive semantic knowledge, thereby enhancing the model’s ability to perform generative recommendation. (2) Our method is inherently plug-and-play, meaning it can directly incorporate the latest advancements from large language models without requiring any modifications to the underlying model architecture. This flexibility allows us to fully leverage the capability of state-of-the-art large language models, leading to improved recommendation performance.

Choice of different quantization methods. To evaluate the impact of different quantization methods on model performance, we conducted a comparative analysis on the Beauty dataset. We apply three different quantization methods: random assignment, RQVAE, and hierarchical K-means to discretize item representations. As illustrated in Figure 8, we observe that hierarchical K-means outperformed the other two methods. Compared to meaningless ID information, hierarchical K-means effectively captures the hierarchical relationship among items, where items sharing the same prefix codes are more similar than different prefix codes and thus more likely to be recommended to users. Additionally, in practice, we find that the RQVAE method suffers from persistent codebook conflicts during quantization, where multiple items are frequently assigned to a same code. To mitigate this issue, following a common practice [29], we append a unique identifier at the end to ensure uniqueness. However, this approach partially disrupts the inherent hierarchical structure of the codebook, which could result in sub-optimal performance in downstream recommendation tasks.

Number of Clusters. We analyzed the effect of the number of clusters K in the hierarchical K-means algorithm on model performance on the Beauty dataset. Specifically, we vary K across 128, 256, and 512. As shown in Figure 9, increasing K from 128 to 256 enhances model performance, likely because a smaller K limits the representational capacity of the codebook, making it hard to assign a distinct unicode to each item. Consequently, many dissimilar items are grouped under similar codes, reducing representational accuracy. When K increases further from 256 to 512, the NDCG@20 metric improves, but the Recall@20 metric declines. This discrepancy may arise due to the larger search space, which increases the complexity of the decoding process. These findings highlight that for generative recommendation systems, selecting an appropriate number of clusters proportional to the total number of items is essential to balancing representation capacity and computational complexity.

5 CONCLUSION AND FUTURE WORK

In this paper, we introduce a novel framework, PRORec, aimed at leveraging a unified code to encode knowledge from both semantic and collaborative modalities in generative recommendation systems. PRORec is composed of three key components: (1) A unified

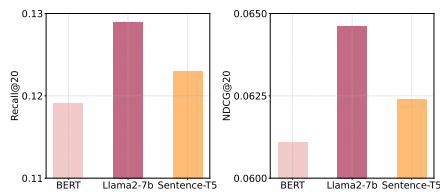


Figure 7: Analysis of different pretrained semantic encoders.

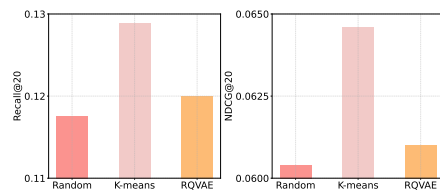


Figure 8: Analysis of different quantization methods.

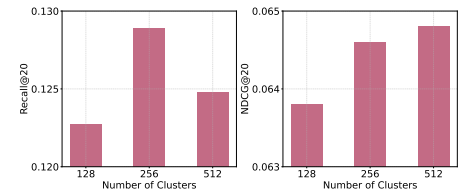


Figure 9: Impact of the number of clusters with K varying from 128 to 512.

code generation framework that integrates semantic and collaborative knowledge, addressing the semantic domination issue by effectively capturing their complementary characteristics. (2) A joint optimization of cross-modality knowledge alignment and next item prediction tasks and a learnable modality adaption layer with AdaLN, which integrate collaborative and semantic knowledge into integrated embeddings, enabling the unified code to represent both modalities information cohesively. (3) An intra-modality knowledge distillation task with a specially designed token, which compensates for the information loss caused by the quantization process and further improves auto-regressive generation quality. Extensive experiments compared with state-of-the-art methods, combined with detailed analyses, validate the effectiveness and robustness of PRORec. In future work, we plan to leverage more available features from the semantic modality, such as item visual features, to further enhance the model's generative recommendation performance.

References

- [1] Alexandr Andoni, Piotr Indyk, and Ilya Razenshteyn. 2018. Approximate nearest neighbor search in high dimensions. In *Proceedings of the International Congress of Mathematicians: Rio de Janeiro 2018*. World Scientific, 3287–3318.
- [2] Sunil Arya, David M Mount, Nathan S Netanyahu, Ruth Silverman, and Angela Y Wu. 1998. An optimal algorithm for approximate nearest neighbor searching fixed dimensions. *Journal of the ACM (JACM)* 45, 6 (1998), 891–923.
- [3] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A simple framework for contrastive learning of visual representations. In *International conference on machine learning*. PMLR, 1597–1607.
- [4] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.
- [5] Jacob Devlin. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [6] Qiming Diao, Minghui Qiu, Chao-Yuan Wu, Alexander J Smola, Jing Jiang, and Chong Wang. 2014. Jointly modeling aspects, ratings and sentiments for movie recommendation (JMARS). In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 193–202.
- [7] Chao Feng, Wuchao Li, Defu Lian, Zheng Liu, and Enhong Chen. 2022. Recommender forest for efficient retrieval. *Advances in Neural Information Processing Systems* 35 (2022), 38912–38924.
- [8] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In *International Conference on Machine Learning*. PMLR, 3887–3896.
- [9] Wei Guo, Hao Wang, Luankang Zhang, Jin Yao Chin, Zhongzhou Liu, Kai Cheng, Qiushi Pan, Yi Quan Lee, Wanqi Xue, Tingjia Shen, et al. 2024. Scaling New Frontiers: Insights into Large Recommendation Models. *arXiv preprint arXiv:2412.00714* (2024).
- [10] Ruining He and Julian McAuley. 2016. Fusing similarity models with markov chains for sparse sequential recommendation. In *2016 IEEE 16th international conference on data mining (ICDM)*. IEEE, 191–200.
- [11] B Hidasi. 2015. Session-based Recommendations with Recurrent Neural Networks. *arXiv preprint arXiv:1511.06939* (2015).
- [12] Dietmar Jannach and Malte Ludewig. 2017. When recurrent neural networks meet the neighborhood for session-based recommendation. In *Proceedings of the eleventh ACM conference on recommender systems*. 306–310.
- [13] Jianchao Ji, Zelong Li, Shuyuan Xu, Wenyue Hua, Yingqiang Ge, Juntao Tan, and Yongfeng Zhang. 2024. Genrec: Large language model for generative recommendation. In *European Conference on Information Retrieval*. Springer, 494–502.
- [14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [15] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.
- [16] Dongmoon Kim, Kun-su Kim, Kyo-Hyun Park, Jee-Hyong Lee, and Keon Myung Lee. 2007. A music recommendation system with a dynamic k-means clustering algorithm. In *Sixth international conference on machine learning and applications (ICMLA 2007)*. IEEE, 399–403.
- [17] Tongyoung Kim, Soojin Yoon, Seongku Kang, Jinyoung Yeo, and Dongha Lee. 2024. SC-Rec: Enhancing Generative Retrieval with Self-Consistent Reranking for Sequential Recommendation. *arXiv preprint arXiv:2408.08686* (2024).
- [18] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. 2022. Autoregressive image generation using residual quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11523–11532.
- [19] JDMCK Lee and K Toutanova. 2018. Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* 3, 8 (2018).
- [20] Xinyu Lin, Wenjie Wang, Yongqi Li, Fuli Feng, See-Kiong Ng, and Tat-Seng Chua. 2024. Bridging items and language: A transition paradigm for large language model-based recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1816–1826.
- [21] Han Liu, Yinwei Wei, Xueming Song, Weili Guan, Yuan-Fang Li, and Liqiang Nie. 2024. MMGR: Multimodal Generative Recommendation with Transformer Model. *arXiv preprint arXiv:2404.16555* (2024).
- [22] Qijiong Liu, Jieming Zhu, Yanting Yang, Quanyu Dai, Zhaocheng Du, Xiao-Ming Wu, Zhou Zhao, Rui Zhang, and Zhenhua Dong. 2024. Multimodal pretraining, adaptation, and generation for recommendation: A survey. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6566–6576.
- [23] Zhongzhou Liu, Hao Zhang, Kuicai Dong, and Yuan Fang. 2024. Collaborative Cross-modal Fusion with Large Language Model for Recommendation. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 1565–1574.
- [24] Chen Ma, Peng Kang, and Xue Liu. 2019. Hierarchical gating networks for sequential recommendation. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 825–833.
- [25] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton Van Den Hengel. 2015. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 43–52.
- [26] Jianmo Ni, Gustavo Hernandez Abrego, Noah Constant, Ji Ma, Keith B Hall, Daniel Cer, and Yinfei Yang. 2021. Sentence-t5: Scalable sentence encoders from pre-trained text-to-text models. *arXiv preprint arXiv:2108.08877* (2021).
- [27] William Peebles and Saining Xie. 2023. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4195–4205.
- [28] Jianpeng Qi, Yanwei Yu, Lihong Wang, Jinglei Liu, and Yingjie Wang. 2017. An effective and efficient hierarchical K-means clustering algorithm. *International Journal of Distributed Sensor Networks* 13, 8 (2017), 1550147717728627.
- [29] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.
- [30] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing personalized markov chains for next-basket recommendation. In *Proceedings of the 19th international conference on World wide web*. 811–820.
- [31] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM international*

- conference on information and knowledge management. 1441–1450.
- [32] Jiayi Tang and Ke Wang. 2018. Personalized top-n sequential recommendation via convolutional sequence embedding. In *Proceedings of the eleventh ACM international conference on web search and data mining*. 565–573.
 - [33] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
 - [34] Jizhe Wang, Pipei Huang, Huan Zhao, Zhibo Zhang, Binqiang Zhao, and Dik Lun Lee. 2018. Billion-scale commodity embedding for e-commerce recommendation in alibaba. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 839–848.
 - [35] Jinpeng Wang, Jieming Zhu, and Xiuqiang He. 2021. Cross-batch negative sampling for training two-tower recommenders. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 1632–1636.
 - [36] Wenjie Wang, Xinyu Lin, Fuli Feng, Xiangnan He, and Tat-Seng Chua. 2023. Generative recommendation: Towards next-generation recommender paradigm. *arXiv preprint arXiv:2304.03516* (2023).
 - [37] Wenjie Wang, Yiyi Xu, Fuli Feng, Xinyu Lin, Xiangnan He, and Tat-Seng Chua. 2023. Diffusion recommender model. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 832–841.
 - [38] Yidan Wang, Zhaochun Ren, Weiwei Sun, Jiyuan Yang, Zhixiang Liang, Xin Chen, Ruobing Xie, Su Yan, Xu Zhang, Pengjie Ren, et al. 2024. Enhanced generative recommendation via content and collaboration integration. *arXiv preprint arXiv:2403.18480* (2024).
 - [39] Ye Wang, Jiahao Xun, Minjie Hong, Jieming Zhu, Tao Jin, Wang Lin, Haoyuan Li, Linjun Li, Yan Xia, Zhou Zhao, et al. 2024. EAGER: Two-Stream Generative Recommender with Behavior-Semantic Collaboration. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3245–3254.
 - [40] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 974–983.
 - [41] Hansi Zeng, Chen Luo, Bowen Jin, Sheikh Muhammad Sarwar, Tianxin Wei, and Hamed Zamani. 2024. Scalable and effective generative information retrieval. In *Proceedings of the ACM on Web Conference 2024*. 1441–1452.
 - [42] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).
 - [43] Tingting Zhang, Pengpeng Zhao, Yanchi Liu, Victor S Sheng, Jiajie Xu, Deqing Wang, Guanfeng Liu, Xiaofang Zhou, et al. 2019. Feature-level deeper self-attention network for sequential recommendation. In *IJCAI*. 4320–4326.
 - [44] Zhe Zhao, Lichan Hong, Li Wei, Jilin Chen, Aniruddh Nath, Shawn Andrews, Aditee Kumthekar, Maheswaran Sathiamoorthy, Xinyang Yi, and Ed Chi. 2019. Recommending what video to watch next: a multitask ranking system. In *Proceedings of the 13th ACM conference on recommender systems*. 43–51.
 - [45] Bowen Zheng, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, Ming Chen, and Ji-Rong Wen. 2024. Adapting large language models by integrating collaborative semantics for recommendation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 1435–1448.
 - [46] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.
 - [47] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 1893–1902.
 - [48] Han Zhu, Daqing Chang, Ziru Xu, Pengye Zhang, Xiang Li, Jie He, Han Li, Jian Xu, and Kun Gai. 2019. Joint optimization of tree-based index and deep model for recommender systems. *Advances in Neural Information Processing Systems* 32 (2019).
 - [49] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning tree-based deep model for recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1079–1088.
 - [50] Jieming Zhu, Mengqun Jin, Qijiong Liu, Zexuan Qiu, Zhenhua Dong, and Xiu Li. 2024. CoST: Contrastive Quantization based Semantic Tokenization for Generative Recommendation. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 969–974.